

Relatorio de Analise Estatica de Malware

Este e um ESQUELETO. Nao escreva nada por cima dos campos; em cada bloco 'Evidencia' rode o comando indicado no terminal e cole a saida no lugar do marcador [COLE A SAIDA AQUI]. As partes de texto marcadas com [Escreva...] sao para voce redigir com suas proprias palavras.

Identificacao da Amostra

Campo	Valor / Evidencia
Nome do arquivo	[Escreva o nome, ex.: mshelp.exe]
Hash SHA-256	[RODE: sha256sum mshelp.exe COLE A SAIDA AQUI]
Hash MD5	[RODE: md5sum mshelp.exe COLE A SAIDA AQUI]
Tamanho (bytes)	[RODE: ls -la mshelp.exe COLE A SAIDA AQUI]
Tipo de arquivo	[RODE: file mshelp.exe COLE A SAIDA AQUI]

1. Sumario Executivo

[Escreva aqui, em 2 a 4 frases, o que a amostra e e faz: tipo de malware, tecnica de execucao, algoritmo de criptografia do payload.]

2. Metodologia e Ambiente

[Escreva: analise 100% estatica (sem execucao), ferramentas usadas (objdump, strings, pefile, Python), e o sistema onde rodou os comandos.]

3. Triage e Arquivos Companheiros

Procure arquivos escondidos no mesmo diretorio (nomes que comecam com '.'). Um arquivo de dados que NAO comeca com os bytes 'MZ' geralmente e payload criptografado.

Passo	Comando	Saida (cole aqui)
Listar tudo (inclui ocultos)	ls -la	[COLE A SAIDA AQUI]
Conferir cabecalho do arquivo de dados	xxd -l 16 <arquivo_companheiro>	[COLE A SAIDA AQUI]

4. Strings

Strings de debug/status costumam narrar o fluxo inteiro do programa (caminhos, nomes de API, mensagens de erro). Rode as duas variantes (ASCII e UTF-16).

Comando	Saida (cole aqui)
strings -a -n 5 mshelp.exe	[COLE A SAIDA AQUI]
strings -a -e l -n 5 mshelp.exe (UTF-16)	[COLE A SAIDA AQUI]

5. Identificacao do Compilador / Empacotador

Procure nas strings a impressao digital do toolchain (ex.: 'FPC 3.2.2 ... for x86_64 - Win64' = Free Pascal/Lazarus; 'FPC_MOVE' tambem indica Free Pascal). Isso define os atalhos das proximas etapas.

Comando	Saida (cole aqui)
strings -a -n 5 mshelp.exe grep -i -E 'FPC Lazarus Free Pascal VMT unwind Go build MSVC'	[COLE A SAIDA AQUI]

[Escreva: qual compilador/linguagem e por que (cite a string exata que voce achou).]

6. Estrutura do PE

Campo	Comando	Valor (cole aqui)
Secoes, entry point, ImageBase	objdump -h mshelp.exe	[COLE A SAIDA AQUI]
TimeStamp	pefile -> pe.FILE_HEADER.TimeDateStamp	[COLE A SAIDA AQUI] (zero = suspeito)
Imports	pefile -> pe.DIRECTORY_ENTRY_IMPORT	[COLE A SAIDA AQUI]

7. Tabela de Simbolos

Rode ANTES de desassemblar na mao. Binarios de FPC/Lazarus costumam ter a tabela de simbolos completa, o que te leva direto ao endereco da chave de criptografia.

Comando	Saida (cole aqui)
objdump -t mshelp.exe grep -i -E 'RC4 KEY IV DECRYPT'	[COLE A SAIDA AQUI]
objdump -t mshelp.exe	[COLE A SAIDA AQUI (ou o trecho relevante)]

8. Desassembly das Funcoes Relevantes

Use objdump com enderecos de inicio/fim. Cole o trecho-chave de cada funcao e uma linha explicando o que ela faz.

8.1 Chamada a RC4DECRYPT (funcao main)

Comando	Saida (cole aqui)
objdump -d -M intel --start-address=0x1000027b0 --stop-address=0x100002820 mshelp.exe	[COLE A SAIDA AQUI]

[Escreva: qual a ordem dos argumentos (rcx, rdx, r8, r9, pilha) e o que cada um recebe (out, Data, &RC4KEY;, RC4IV0, RC4IV1).]

8.2 Montagem da chave (3 x FPC_MOVE)

Comando	Saida (cole aqui)
objdump -d -M intel --start-address=0x10002ee50 --stop-address=0x10002ef60 mshelp.exe	[COLE A SAIDA AQUI]

[Escreva: como os 24 bytes sao montados (RC4KEY 16 || RC4IV0 4 || RC4IV1 4) e o SetLength(..., 0x18).]

8.3 Nucleo RC4 (KSA + PRGA)

Comando	Saida (cole aqui)
objdump -d -M intel --start-address=0x10002ecb0 --stop-address=0x10002ee30 mshelp.exe	[COLE A SAIDA AQUI]

[Escreva: confirme que e RC4 padrao, apontando o KSA ($S[i]=i$, $j=(j+S[i]+chave[i \bmod len]) \& 0xFF$, swap) e o PRGA ($i=i+1$, $j=S[i]$, swap, $saida = S[(S[i]+S[j]) \& 0xFF] \text{ XOR byte}$).]

9. Recuperacao da Criptografia

Chave final (24 bytes) = RC4KEY(16) || RC4IV0(4) || RC4IV1(4). Pegue os valores exatos da tabela de simbolos ou do .data.

Material	Valor (hex, cole aqui)
RC4KEY (16 bytes)	[COLE AQUI]
RC4IV0 (4 bytes)	[COLE AQUI]
RC4IV1 (4 bytes)	[COLE AQUI]
Chave final (24 bytes)	[COLE AQUI]

10. Extracao do Payload

Escreva um script de descriptografia em Python (RC4 com a chave de 24 bytes) e rode sobre o arquivo de dados. Verifique que a saida começa com 'MZ' e e um PE valido.

Passo	Comando / Acao	Resultado (cole aqui)
Executar script de decrypt	python3 decrypt.py <arquivo_companheiro> <saida.bin>	[COLE A SAIDA AQUI]
Verificar cabecalho MZ	xxd -l 2 saida.bin	[COLE A SAIDA AQUI] (esperado: 4d)
Validar PE	python3 -c "import pefile; pe=pefile.PE('saida.bin'); print(pe.is_valid())"	[COLE A SAIDA AQUI]
Calcular hash do payload	sha256sum saida.bin	[COLE A SAIDA AQUI]

11. Técnica de Execução

[Escreva: cruze os nomes de API achados nas strings/imports com a técnica (process hollowing, DLL injection, etc.) e cite as APIs que comprovam.]

12. Indicadores de Comprometimento (IOCs)

Tipo	Valor (cole aqui)
Hash SHA-256 da amostra	[COLE AQUI]
Hash SHA-256 do payload extraído	[COLE AQUI]
Nomes de arquivo (dropper/companheiro)	[COLE AQUI]
Strings / caminhos suspeitos	[COLE AQUI]

13. Conclusão

[Escreva: resumo dos achados, o que o malware faz, e limitações da análise estática (ex.: não confirmou comportamento em runtime).]